1 - Modelo (Persistência)

String, Objetos – Não precisa informar nullable (?)

DateTime, Int16,Int32, Int64, Double, Enum - Precisa informar nullable (?)

2- Map (Mapeamento da Classe)

- Não precisa mapear o ID

- Os campos da tabela sempre em caixa alta devido ao MYSQL (CASE SENSITIVE)

- Para Enum

```
        Map(x => x.Tipo_Movimento,  "TIPO_MOVIMENTO" ).CustomType<Tipo_Movimento_Conta_Financeira>
();
```

```
- Objetos Agregados
      References(x => x.Conta_Contabil,  "ID_CONTA_CONTABIL" );
- String (Tem q informar o Length)
      Map(x => x.Codigo,  "CODIGO" ).Length(20);
- String(Blob – Tem q informar o Lenght)
      Map(x => x.Codigo,  "CODIGO" ).Length( 5000 );
```

- Double, DateTime, Int16, Int32, Int64 (Não precisa de Lenght)

```
      Map(x => x.Valor,  "VALOR" );
- Listas
      HasMany(x => x.Perguntas).KeyColumn( "ID_SCRIPT" ).Cascade.All().Inverse();
```

3- Projeto Web (Connect / Financeiro)

3.1 - ModelView

Reflete o modelo da view. Não precisa seguir o modelo de persistência.

Ex.: Na View tem CheckBox... O modelo do razor pra check pra check box dá problema com enum.

Basta colocar no modelo de view o boolean para resolver este problema. E no bind da Api fazer a mudança de tipo.

Nome do arquivo com os modelos de View sempre com “Models” no final. (Ex. Conta_FinanceiraModels)

3.2 - ListaModel

Classes de Lista sempre com “ListaModel” no final. (Ex. Conta_FinanceiraListaModel)

3.3 - CadastroModel

Classes de Cadastro sempre com “CadastroModel” no final. (Ex. Conta_FinanceiroCadastroModel)

Não esquecer : Caso o cadastro utilize a estrutura padrão utilizar a interface `IViewModel<Int64>`

3.4 - DataTable

Classes de Modelo de DataTable sempre com “_DataTableRequest” no final. (Ex. Conta_Financeira_DataTableRequest)

4 - WebAPI

Padrão: Sempre com “ApiController” no final (Ex.: Conta_FinanceiraApiController)

4.1 - GetAll

É um método assíncrono consumido pelo DataTable via ajax, passando um objeto com parâmetros do datatable e retornando objeto esperado pelo DataTable.

O que é método assíncrono.

São métodos onde o sistema operacional abre uma outra thread (não trava o processo esperando o termino).

As threads são uteis por utilizar melhor o processamento do servidor, balanceando o processamento.

Ex.: async Task<DataTableResponse<Conta_FinanceiraListaModel>> GetAll(Conta_Financeira_DataTableRequest request)

Obs.: Foi utilizado DAO para consumo do banco, mas utilizando a conexão do Hibernate como intermediária. Para evitar construção de estruturas em paralelo, abertura de múltiplas conexões simultâneas e outros.

5- View de Listagem

Padrão: Index.cshtml

Cuidado com o arquivo Index , tem vários index no projeto

Cuidado com o resouces:

Utilizar
no
Connect**Resources** para nomes em comum. Genéricos!

RsCadastros para Connect

RsConfiguracao para as configurações Connect (Deprecated)

RsRadius para o Radius.

RsFinanceiro para Financeiro

Toda tela vem com o padrão: checkboxs para seleção, filtro (search), excluir, ativar e desativar.

Filtro adicionais serão acionados via modal em casos específicos (quando necessário)...

Procedimento de construção padrão

Criar com base em alguma tela simples (Ex.: Listagem de Banco)

Renomear com muito cuidado, selecionando documento atual, os Termos de Banco para o Novo Objeto. Ex.: Banco para Conta Financeira.

Conferir os objetos.

Mudar colunas para as colunas do projeto.

O Web Api converto json para camelcase deixando a primeira letra sempre em minisculo. Cuido.

Como padrão toda listagem terá as funcionalidade:

AlterarStatusObj, txtSearch, btnInativar, btnAtiva e btnExcluir.

Mas não é necessário sempre utilizar todas.

Caso não apareça a informação do datatable Nenhum registro encontrado é porque está com erro. Verifique o console.

6 – CADASTRO

Padrão: Cadastro.cshtml

Cuidado com o arquivo Cadastro , tem vários index no projeto

Cuidado com o resouces:

Utilizar
no
Connect:**Resources** para nomes em comum. Genéricos!

RsCadastros para Connect

RsConfiguracao para as configurações Connect (Deprecated)

RsRadius para o Radius.

RsFinanceiro para Financeiro

6.1 - Labels

Os labels por convenção ficam na parte superior do controle.

Tem duas formas de faze-lo:

Com Tag html

```
<label for="iNomeImpresso" class="mr-3">@RsFinanceiro.NOMEIMPRESSOCARTAO:</label>
```

Com Razor

```
@Html.DisplayFor(model => model.Nome)
```

6.1.1 - Controles String

```
@Html.TextBoxFor(model => model.NomeImpresso, new { @class = "form-control", @id =
"iNomeImpresso", @autocomplete = "off" })
```

Atenção Sempre Colocar a Tag de Validação em todos os controles simples (não listas e não booleans)

```
@Html.ValidationMessageFor(model => model.NomeImpresso, "", new { @class = "text-danger" })
```

Para incluir mascaras

```
$("#iCPF").mask("000.000.000-00", { reverse: true });
$("#iCNPJ").mask("00.000.000/0000-00", { reverse: true });
```

6.1.2 – Controles Date (Colocar o ValidationMessageFor)

@Html.TextBoxFor(m => m.DataNascimento, new { @class = "form-control datepicker", @data_provide = "datepicker", @id = "iNascimento", @autocomplete = "off" })

6.1.3 – Enum (Colocar o ValidationMessageFor)

@Html.EnumDropDownListFor(m => m.Tipo_Resposta, Resources.SELECIONE, new { @class = "form-control", @id = "iTipo_Resposta", @autocomplete = "off" })

6.1.4 – Blob (Colocar o ValidationMessageFor)

@Html.TextAreaFor(model => model.Explicacao_Cliente, new { @class = "form-control", @id = "iExplicacao_Cliente", @autocomplete = "off", @required = true })

6.1.5 – Checkbox

@Html.CheckBoxFor(m => m.Status, new { @class = "form-control", @id = "iStatus", @name = "iStatus", @data_toggle = "toggle", @data_on = **"Ativo"** , @data_off = **"Inativo"** , @data_onstyle = "success", @data_offstyle = "light" })

6.1.6 – Combobox

@Html.DropDownListFor(m => m.Id_Script, new SelectList(ViewBag.Scripts, "Id", "Nome"), Resources.SELECIONE, new { @class = "form-control", @id = "iPrioridade", @autocomplete = "off" })

6.1.7 – Listagem (Olhar Script_Atendimento)

6.18 -

```
< div class ="col-sm-2">
@ if (Model.Id != 0)
{
   < div class ="form-group">
< label for ="iStatus" class ="mr-3">
@ RsFinanceiro.STATUS: </ label >< br >
    @ Html.CheckBoxFor(model => model.Status,  new { @class =  "form-control" , @id =  "iStatus" ,
@name =  "iStatus" , @data_toggle =  "toggle" , @data_on =  "Ativo" , @data_off =  "Inativo" ,
@data_onstyle =  "success" , @data_offstyle =  "light" })
                          </ div >
  }
      </ div >
```

6.19 – mascara , usar o doc connect.js

```
//MaskIP
    $( ".maskip" ).mask( "000.000.000.000" );
```